

INTERNSHIP PROGRAM

Weekly Standup

Presented by

Shatakshi

Under the guidance of

Sai Prasad

Senior DevOps Engineer

Employee Management System

Introduction

This project is a simple employee management system built using AWS serverless services. Users can add employee details through a website, and the data is processed and stored automatically using an event-driven backend. No servers are managed manually — everything runs on managed AWS services.

Architecture Diagram

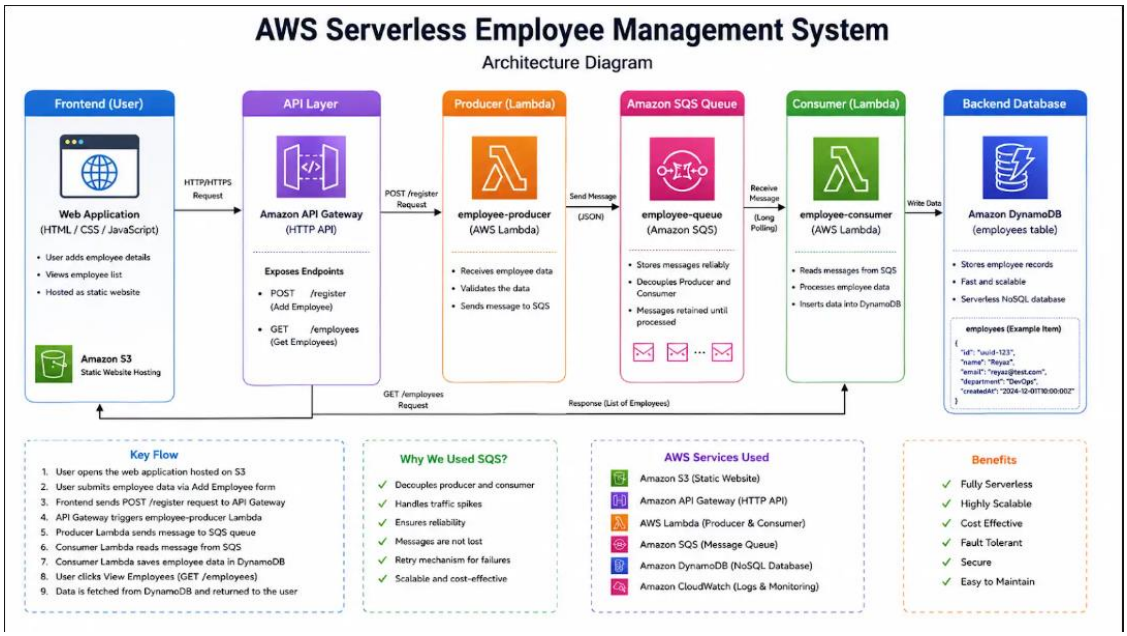


Fig 1: End-to-end AWS architecture flow

Architecture Flow

The system follows this flow:

1. User opens the website hosted on Amazon S3.
2. User fills in employee details and clicks 'Add Employee'.
3. The frontend sends a POST request to API Gateway (/register).
4. API Gateway triggers the Producer Lambda function.
5. Producer Lambda sends the employee data to an SQS queue.
6. SQS automatically triggers the Consumer Lambda function.
7. Consumer Lambda reads the message and saves the data into DynamoDB.
8. The frontend calls GET /employees to fetch all records.
9. A Lambda function reads the data from DynamoDB and returns it.
10. The employee list is displayed in a table on the website.

AWS Services Used

Service	Purpose
Amazon S3	Hosts the frontend website (HTML/CSS/JS)
API Gateway	Exposes REST APIs (POST /register, GET /employees)
AWS Lambda	Runs backend logic (Producer, Consumer, Fetch)
Amazon SQS	Queues employee data for async processing
DynamoDB	Stores employee records (NoSQL database)
CloudWatch	Monitoring and logs

About Amazon SQS

Amazon SQS (Simple Queue Service) is a fully managed message queue used to connect the Producer and Consumer Lambda functions. Instead of the Producer writing directly to DynamoDB, it simply drops the employee data into the queue as a message. The Consumer Lambda then picks up messages from the queue and saves them into DynamoDB. This keeps the two functions decoupled, absorbs sudden spikes in traffic, and makes sure no employee data is lost even if the Consumer is temporarily slow or unavailable.

Working Application

Screenshot of the deployed website hosted on Amazon S3, showing an employee being added and the records fetched from DynamoDB:

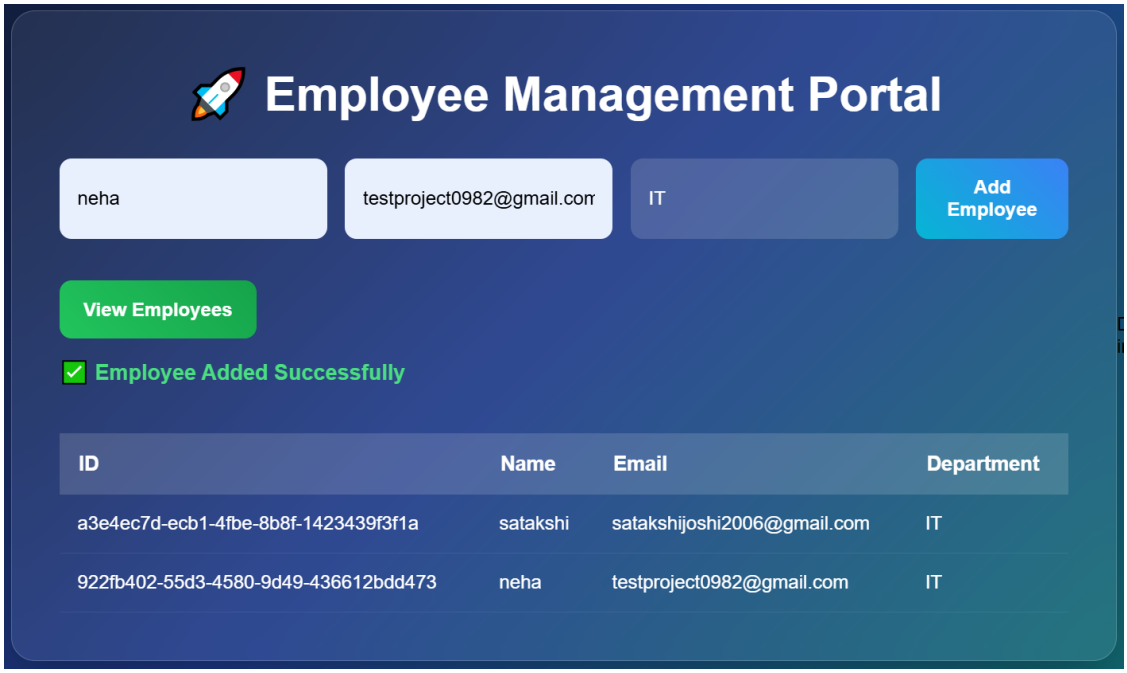


Fig 2: Employee Management Portal

Deployment Link: http://employe-bucket-0416.s3-website.ap-south-1.amazonaws.com/

INTRODUCTION TO DEVOPS

DevOps is a culture and set of practices that brings software development and IT operations together to deliver applications faster, more reliably, and with continuous feedback.

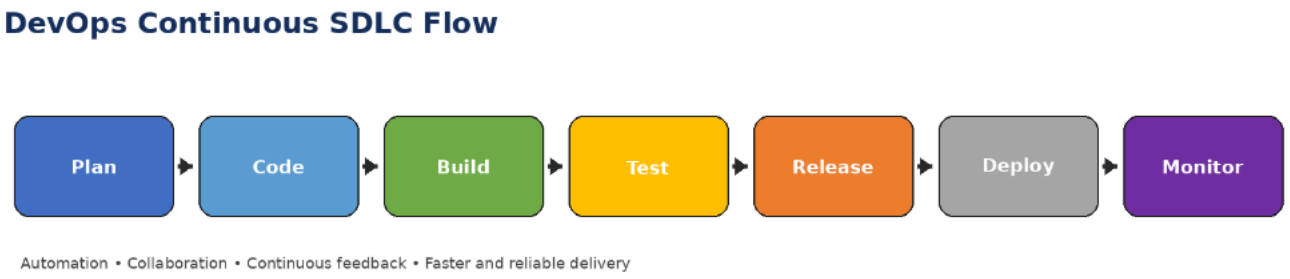
1. What is DevOps?

DevOps is a combination of Development and Operations. It encourages teams to work together throughout the software development life cycle (SDLC), instead of treating coding, testing, deployment, and support as separate activities. DevOps uses automation, collaboration, monitoring, and continuous improvement to reduce delays and improve software quality.

2.why devops is important ?

Modern software development follows **Agile methodologies**, where requirements and features are developed and delivered in **short, frequent iterations** rather than through one large release. Because development happens continuously and changes need to reach users quickly, traditional manual build, testing, and deployment processes become slow and error-prone. This created the need for **CI/CD (Continuous Integration and Continuous Delivery/Deployment)**, which is a key practice of DevOps

3. DevOps and the SDLC



4. Common DevOps Tools and Their Purpose

SDLC Area	Example Tools	Purpose
Code Repository	Git, GitHub	Store source code, manage versions, and collaborate.
Build	Maven, POM.xml	Compile code and download project dependencies.
Test & Quality	SonarQube	Perform code analysis, quality checks, and security scanning.
Artifacts	Nexus, Amazon S3	Store and manage build outputs and release packages.
CI/CD	Jenkins, GitHub Actions, AWS CodePipeline	Automate integration, testing, and delivery.
Containers	Docker, ECS, Fargate	Package applications with their dependencies.
Orchestration	Kubernetes, EKS	Manage containers, scaling, and service availability.
Infrastructure as Code	Terraform, Ansible	Provision and configure infrastructure consistently.
Monitoring	Prometheus, Grafana	Track metrics, visualize performance, and detect problems.

5. Key DevOps Practices

✓ **Continuous Integration (CI): Developers frequently merge code into a shared repository. Automated builds and tests validate every change.**

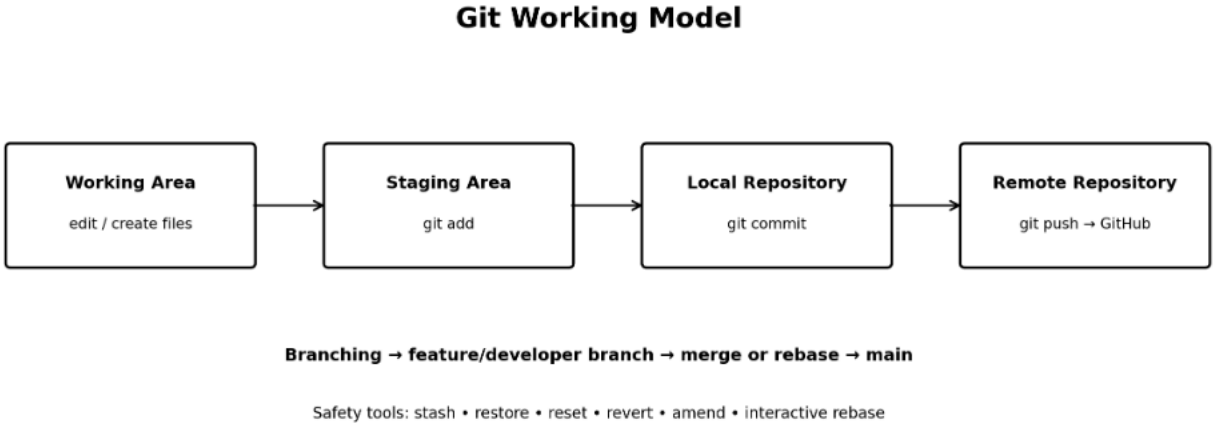
✓ **Continuous Delivery / Deployment (CD): Validated software is packaged and released through an automated pipeline. Continuous deployment can automatically move approved changes into production.**

✓ **Infrastructure as Code (IaC): Infrastructure is defined in files and managed through tools such as Terraform, making environments repeatable.**

✓ **Containerization: Docker packages an application and its dependencies so it runs consistently across environments.**

✓ **Monitoring and Feedback: Metrics, logs, alerts, and dashboards provide visibility into application and infrastructure health.**

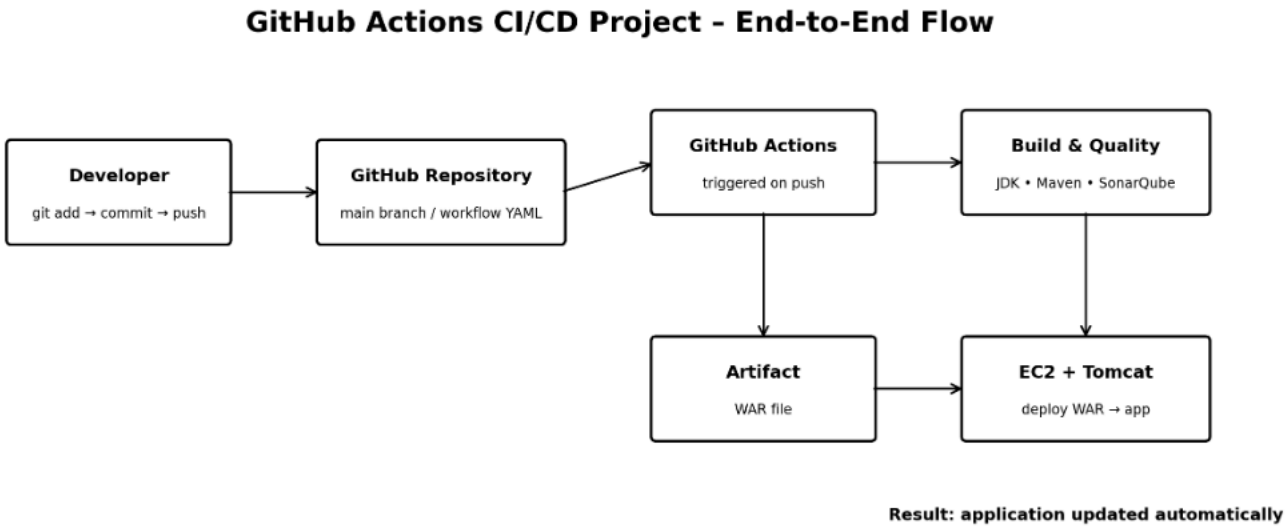
Git & GitHub – Version Control Workflow



Core lifecycle

- **git init creates the local repository and .git directory; git status shows the current state.**
- **Typical flow: create/modify → git status → git add → git commit → git push to GitHub.**
- **git fetch downloads remote changes without integrating them; git pull downloads and integrates them.**
- **git clone downloads a complete remote repository to a local directory.**
- **Authentication covered: HTTPS with a personal access token and SSH key authentication.**

- **3. GitHub Actions CI/CD – Java + Maven + Tomcat**



GIT & GITHUB

1. What is Git & What is GitHub?

Git

Git is a free, open-source Distributed Version Control System (DVCS) used to track changes in source code during software development. It allows multiple developers to work on the same project simultaneously without overwriting each other's work, and keeps a complete history of every change ever made.

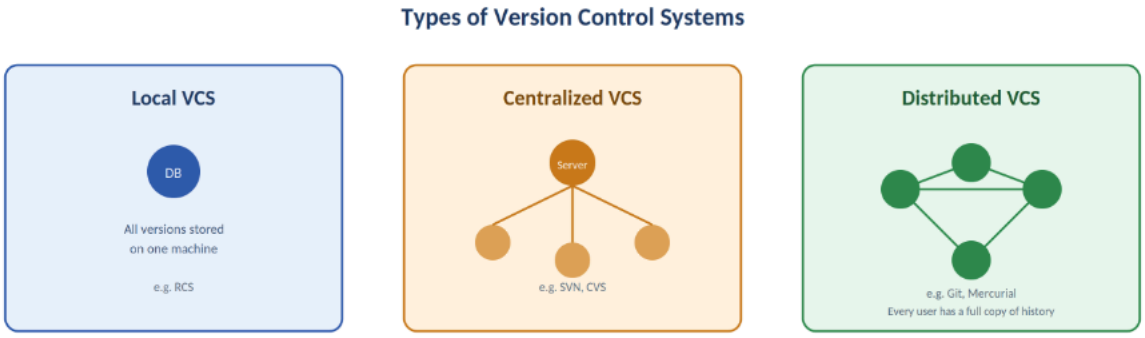
GitHub

GitHub is a cloud-based hosting platform for Git repositories. It adds collaboration features on top of Git — Pull Requests, Issues, code review, project boards, and CI/CD pipelines (GitHub Actions) — making it the most popular place for teams to store and collaborate on Git projects. Git is the tool; GitHub is a service built around it (GitLab and Bitbucket are similar alternatives).

2. Types of Version Control Systems

Version control systems have evolved through three broad generations:

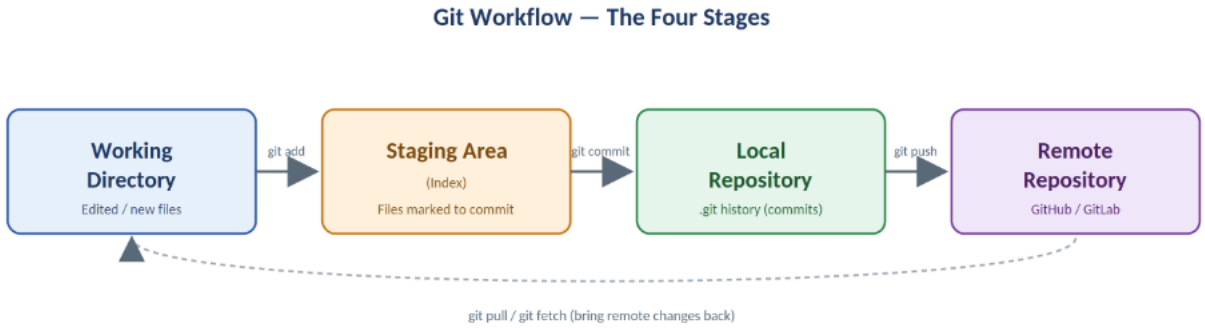
- Local VCS: Keeps every version of a file on the same local machine, in a simple local database. Easy to lose everything if the disk fails.
- Centralized VCS (CVCS): A single central server holds all versioned files (e.g. SVN, CVS). Everyone commits to one server — simple, but a single point of failure and requires network access.
- Distributed VCS (DVCS): Every contributor clones a full copy of the repository, including its entire history (e.g. Git, Mercurial). Work can happen offline, and there is no single point of failure.



3. The Git Workflow — Types of Stages

A file in Git moves through four distinct stages as it goes from an edit on disk to a permanent, shared record of history:

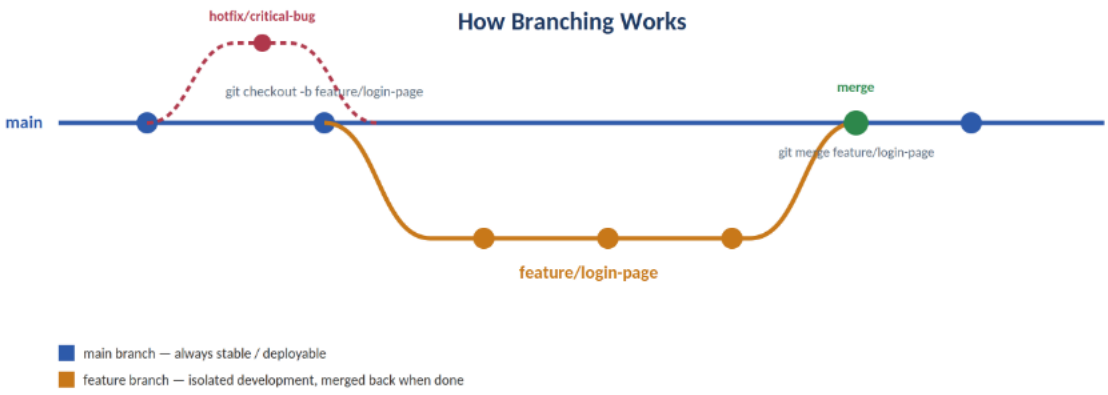
- Working Directory: Where you actually edit files. Git sees these as "modified" or "untracked".
- Staging Area (Index): A holding area where you choose exactly which changes will go into the next commit, using git add.
- Local Repository: Once committed (git commit), a snapshot is permanently saved to the local .git history.
- Remote Repository: Pushing (git push) uploads local commits to a shared server such as GitHub, making them available to the team.



4. Branching

A branch is simply a movable pointer to a commit. Branching lets developers diverge from the main line of development to work on a feature, fix, or experiment in isolation — without affecting the stable main codebase — and later merge that work back in.

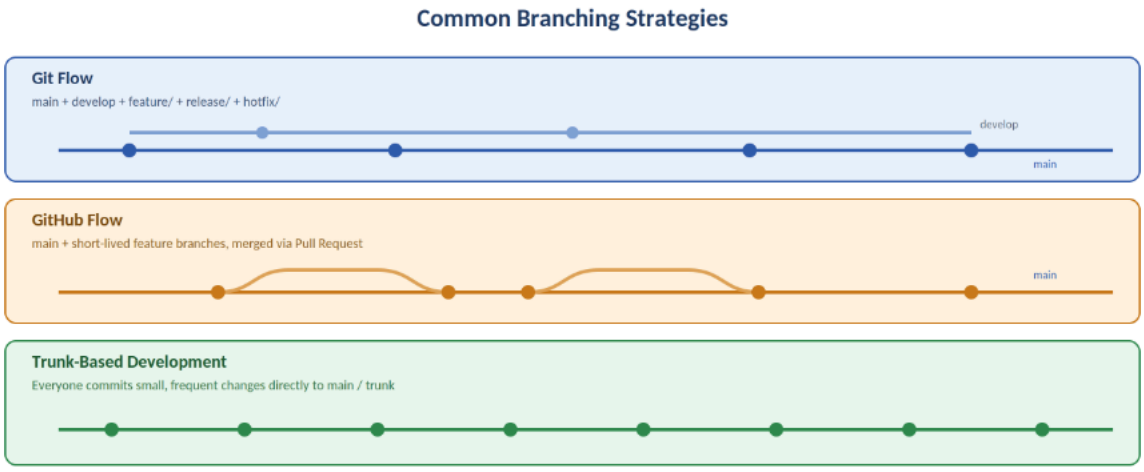
- main / master: The primary branch, usually kept stable and deployable at all times.
- feature branches: Created off main for a single feature or task, merged back once complete.
- hotfix branches: Short-lived branches created to urgently patch a bug in production.


How Branching Works
hotfix/critical-bug
git checkout -b feature/login-page
main
merge
git merge feature/login-page
feature/login-page
main branch — always stable / deployable
feature branch — isolated development, merged back when done

5. Branching Strategies

A branching strategy is a team's agreed convention for how branches are created, named, and merged. The right choice depends on release cadence and team size.

- Git Flow: main + develop + feature/ + release/ + hotfix/ branches. Provides strong structure for projects with scheduled releases, at the cost of more overhead.
- GitHub Flow: A single main branch plus short-lived feature branches merged back via Pull Requests after review. Simple and ideal for continuous deployment.
- Trunk-Based Development: Everyone commits small, frequent changes directly to main (or very short-lived branches), relying on feature flags for incomplete work. Favoured by high-velocity teams practising CI/CD.



6. Important Git Commands

Category	Command	What it does
Setup	git init	Initialize a new Git repository
	git clone <url>	Copy a remote repository to local machine
	git config --global user.name "Name"	Set the committer name globally
Staging & Committing	git status	Show state of working directory & staging area
	git add <file>	Move changes to the staging area

Category	Command	What it does
	git commit -m "msg"	Save staged changes to local history
Branching & Merging	git branch	List, create, or delete branches
	git checkout -b <name>	Create and switch to a new branch
	git merge <branch>	Merge a branch into the current branch
	git rebase <branch>	Reapply commits on top of another base
Remote & Sync	git push	Upload local commits to the remote repository
	git pull	Fetch and merge remote changes into current branch
	git fetch	Download remote history without merging
History & Undo	git log	View commit history
	git diff	Show changes not yet staged
	git reset <file>	Unstage a file, keep the changes
	git revert <commit>	Create a new commit that undoes a prior one

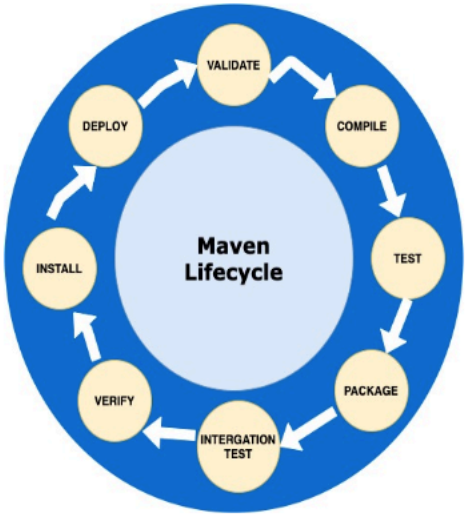
MAVEN

Introduction:

Apache Maven is a popular build automation and project management tool mainly used for Java projects. It helps developers manage the entire project build process, including compiling source code, running tests, packaging applications, and creating deployable files such as JAR and WAR files. Maven uses a configuration file called pom.xml (Project Object Model), where project information, dependencies, plugins, and build settings are defined.

Maven build lifecycle:

The **Maven Build Lifecycle** is a sequence of predefined phases that Maven follows to build and manage a Java project. Each phase performs a specific task, and executing a later phase automatically executes all the earlier phases



The main Maven build lifecycle phases are:

1. Validate – Checks whether the project is correctly configured and all required information is available.
2. Compile – Compiles the Java source code into .class files.
3. Test – Runs the unit tests of the project.
4. Package – Packages the compiled code into a distributable format such as a JAR or WAR file.
5. Install – Installs the generated package into the local Maven repository
6. Deploy – Uploads the final package to a remote Maven repository, where it can be shared with other developers or build systems.

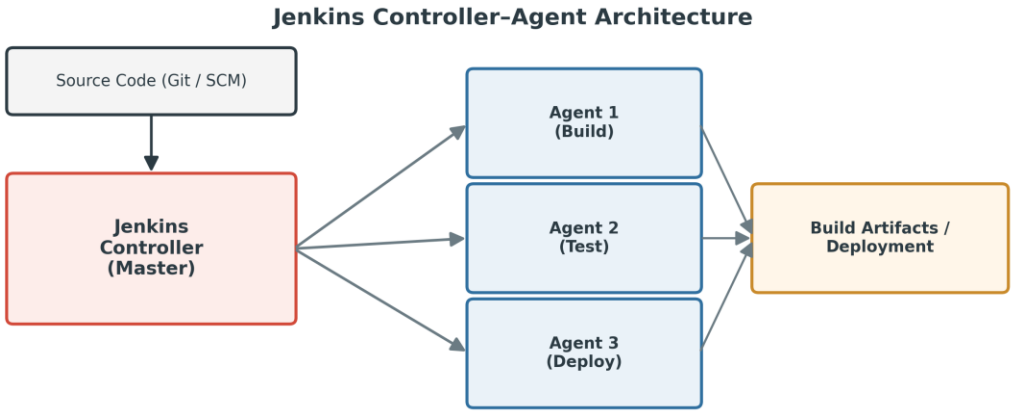
JENKINS

1. Introduction to Jenkins

Jenkins is a free and open-source automation server used to build, test, and deploy software reliably and consistently. It enables teams to practice Continuous Integration (CI) and Continuous Delivery/Deployment (CD) by automating repetitive parts of the software development lifecycle.

2. Jenkins Architecture (Controller–Agent Model)

Jenkins follows a Controller–Agent (Master–Slave) architecture. The Controller manages the overall configuration, scheduling, and user interface, while Agents are worker nodes that execute the actual build, test, and deployment tasks. This distributed model allows workloads to be balanced across multiple environments and platforms.



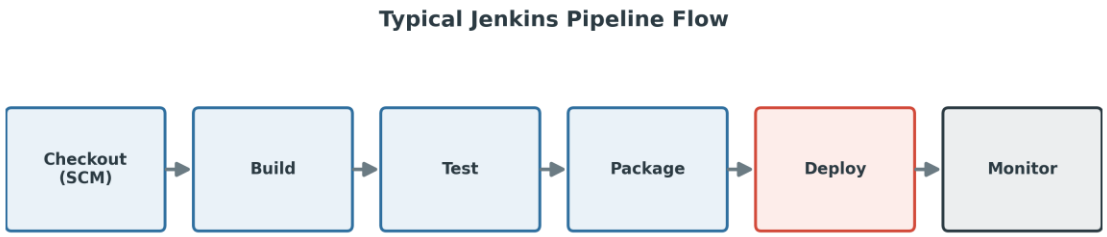
3. Jenkins Pipelines

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines directly into Jenkins. It defines the entire build process as code — typically written in a file called a Jenkinsfile — covering stages such as build, test, and deploy. Pipelines can be written in two styles:

- Declarative Pipeline – a simplified, structured syntax that is easier to read and maintain.
- Scripted Pipeline – a more flexible, Groovy-based syntax offering fine-grained control.

Defining pipelines as code (Pipeline-as-Code) allows version control, code review, and reuse of the delivery process, making automation reliable, repeatable, and auditable.

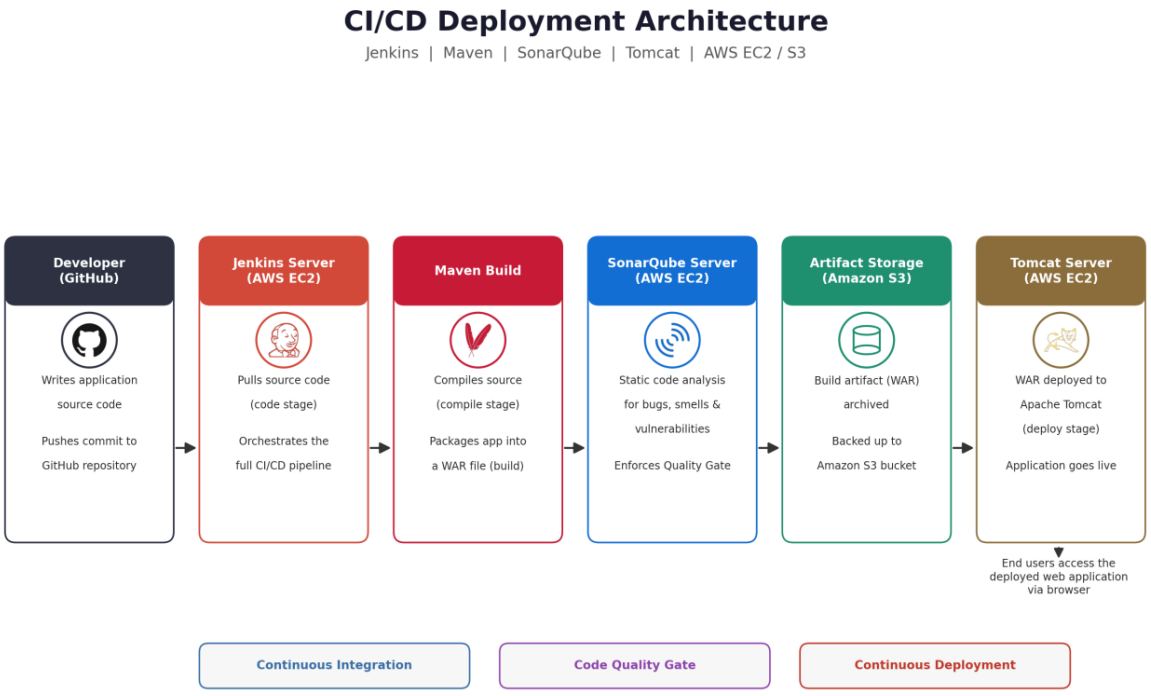
4. Typical Pipeline Flow



PROJECT REPORT

End-to-End CI/CD Deployment Pipeline

Using Jenkins, Maven, SonarQube, Apache Tomcat and AWS (EC2 / S3)



Automated build, code-quality analysis and deployment of a Java web application

1. Introduction

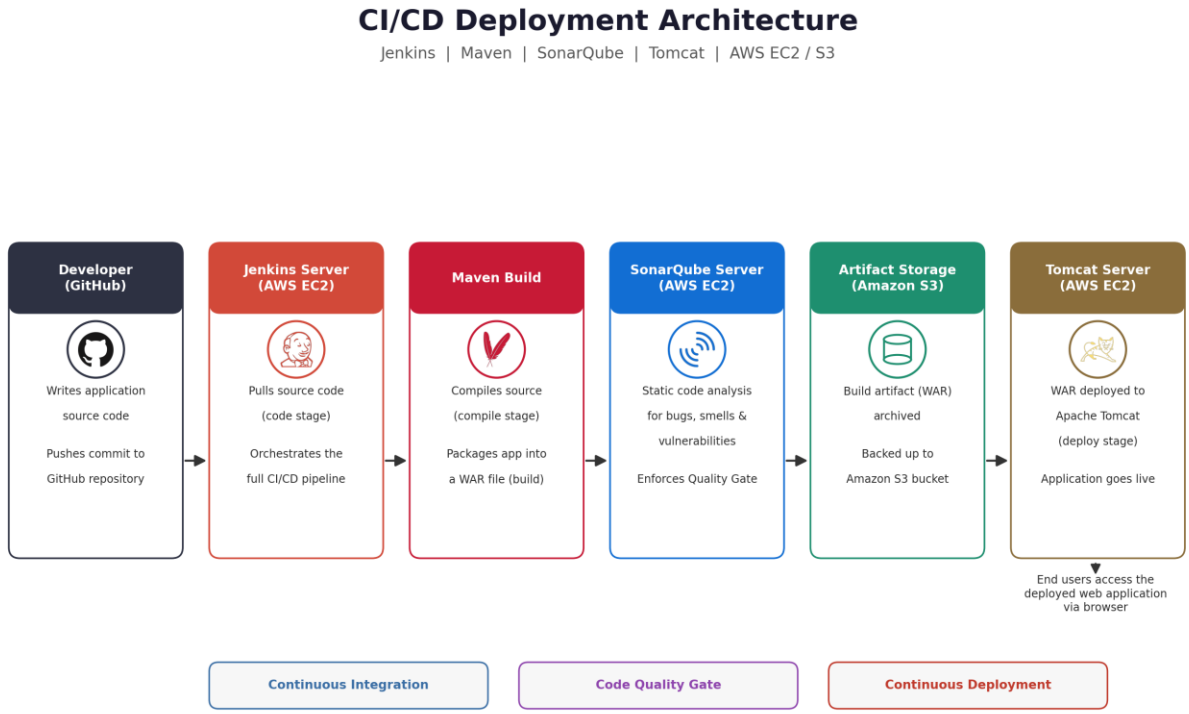
This project implements a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Java-based web application. The pipeline automates every step from source code check-out to production deployment, removing the need for any manual build or release activity.

Jenkins acts as the automation server and orchestrates the pipeline. Apache Maven compiles the source code and packages it into a deployable WAR file. SonarQube performs static code analysis to catch bugs, code smells and vulnerabilities before the build is promoted, enforcing a Quality Gate. The final artifact is deployed to an Apache Tomcat server running on an AWS EC2 instance, making the application publicly accessible. All supporting infrastructure — the Jenkins server, the SonarQube server and the Tomcat application server — is hosted on separate AWS EC2 instances, with Amazon S3 used for artifact / static storage.

The application deployed through this pipeline is a Disney+ Hotstar–style streaming web front end, used here as the sample workload for the pipeline.

2. System Architecture

The diagram below shows the overall flow of the system, from the developer's commit to the application being served to end users.



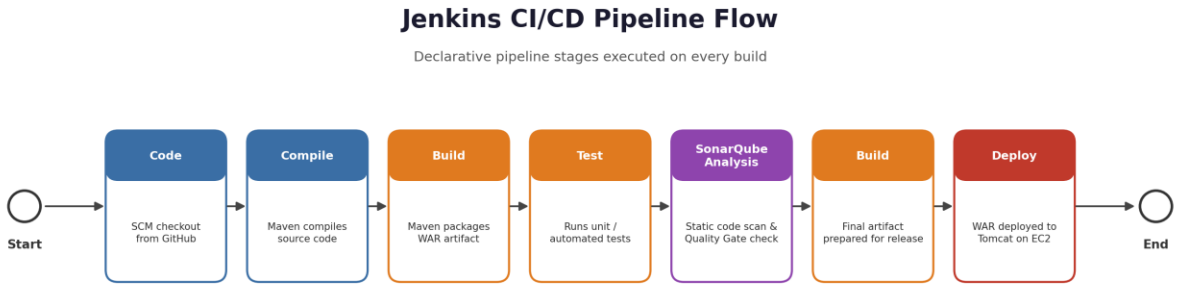
2.1 Architecture Flow

1. Developer writes and commits the application source code to a GitHub repository.
2. Jenkins, running on an AWS EC2 instance, checks out the latest code (Code stage).
3. Maven compiles the source code (Compile stage).

4. Maven builds and packages the application into a WAR file (Build stage).
5. Automated tests are executed against the build (Test stage).
6. SonarQube analyses the code for bugs, vulnerabilities, code smells and coverage, and evaluates the Quality Gate (SonarQube Analysis stage).
7. On passing the Quality Gate, the final deployable artifact is built and archived, and a copy is stored in Amazon S3 (Build stage).
8. Jenkins deploys the WAR file to the Apache Tomcat server running on a separate EC2 instance (Deploy stage).
9. End users access the live application through the browser, over the Tomcat server's public IP and port.

3. Jenkins CI/CD Pipeline

The Jenkins pipeline is written as a declarative pipeline consisting of seven stages. Each stage is visible on the Jenkins “Stages” view, along with its execution status and duration for every build, giving full visibility into the automation process.



3.1 Stage Details

Stage	Purpose
Code	Checks out the latest source code from the GitHub repository.
Compile	Compiles the Java source code using Maven.
Build	Packages the compiled code into a WAR artifact.
Test	Executes automated / unit tests against the build.
SonarQube Analysis	Runs static code analysis and checks the SonarQube Quality Gate.
Build	Finalises and archives the deployable artifact.
Deploy	Deploys the WAR file to the Apache Tomcat server on EC2.

4. Tools & Technologies Used

Tool / Service	Role in the Project
GitHub	Source code repository and version control.
Jenkins	CI/CD automation server that orchestrates the pipeline.
Apache Maven	Builds, compiles and packages the Java application (WAR).
SonarQube	Static code analysis and code-quality Quality Gate.
Apache Tomcat	Application server that hosts and runs the deployed WAR file.
AWS EC2	Virtual machines hosting Jenkins, SonarQube and Tomcat.
Amazon S3	Storage for build artifacts / static assets.

5. About the Key Tools

5.1 Jenkins

Jenkins is an open-source automation server used to build, test and deploy software continuously. It runs pipelines defined as code (Jenkinsfile), which makes the build process repeatable, version-controlled and easy to audit. In this project, Jenkins connects the source repository, the build tool, the code-quality server and the deployment target into a single automated workflow.

5.2 Apache Maven

Maven is a build automation tool used primarily for Java projects. It manages project dependencies, compiles source code, runs tests and packages the application into a distributable artifact (a WAR file for a web application). Maven's standardised project structure and lifecycle (compile → test → package) make it straightforward to plug into a Jenkins pipeline.

5.3 SonarQube

SonarQube is a self-hosted platform for continuous code-quality inspection. It performs static analysis of the source code to detect bugs, vulnerabilities, security hotspots and code smells, and reports metrics such as coverage and duplication. A Quality Gate defines pass/fail conditions (for example, zero new bugs or a minimum coverage percentage); if a build does not meet the gate, the pipeline can be configured to stop before deployment. This keeps poor-quality or insecure code from reaching production.

5.4 Apache Tomcat

Apache Tomcat is an open-source web server and servlet container that runs Java web applications packaged as WAR files. In this project, Jenkins deploys the built WAR artifact directly to Tomcat's webapps directory on an EC2 instance, after which the application becomes accessible over HTTP on the configured port.

6. Working Application

6.1 Jenkins Pipeline Stage View



Fig 3: Jenkins pipeline stage view across recent builds

6.2 Jenkins Job Status — SonarQube Quality Gate

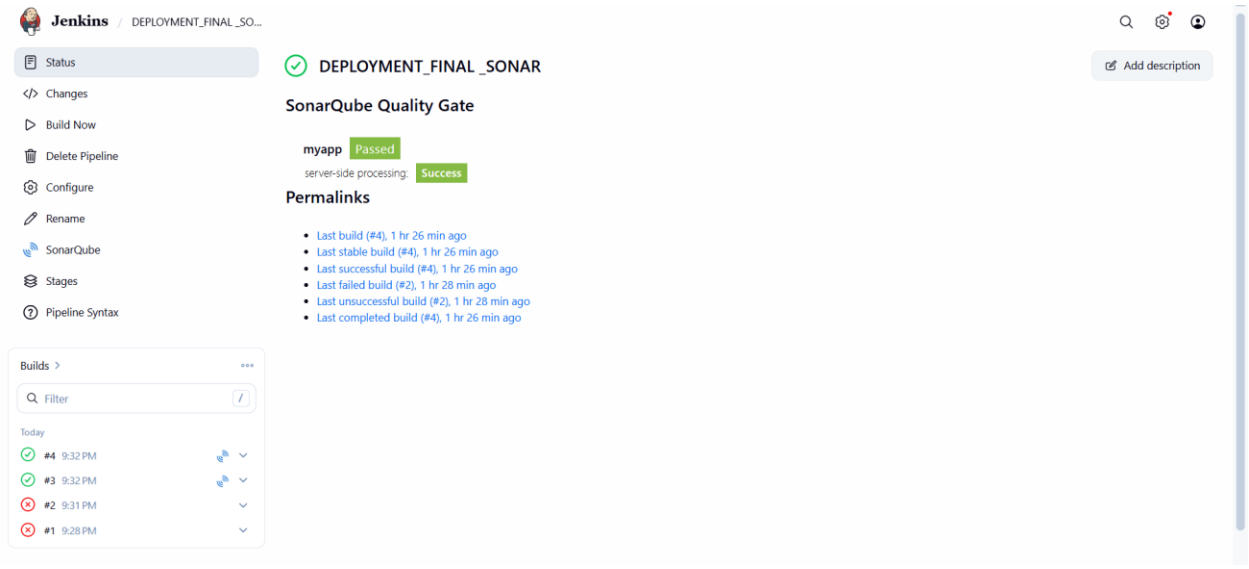


Fig 4: Jenkins job status showing the SonarQube Quality Gate result and build permalinks

6.3 SonarQube Dashboard

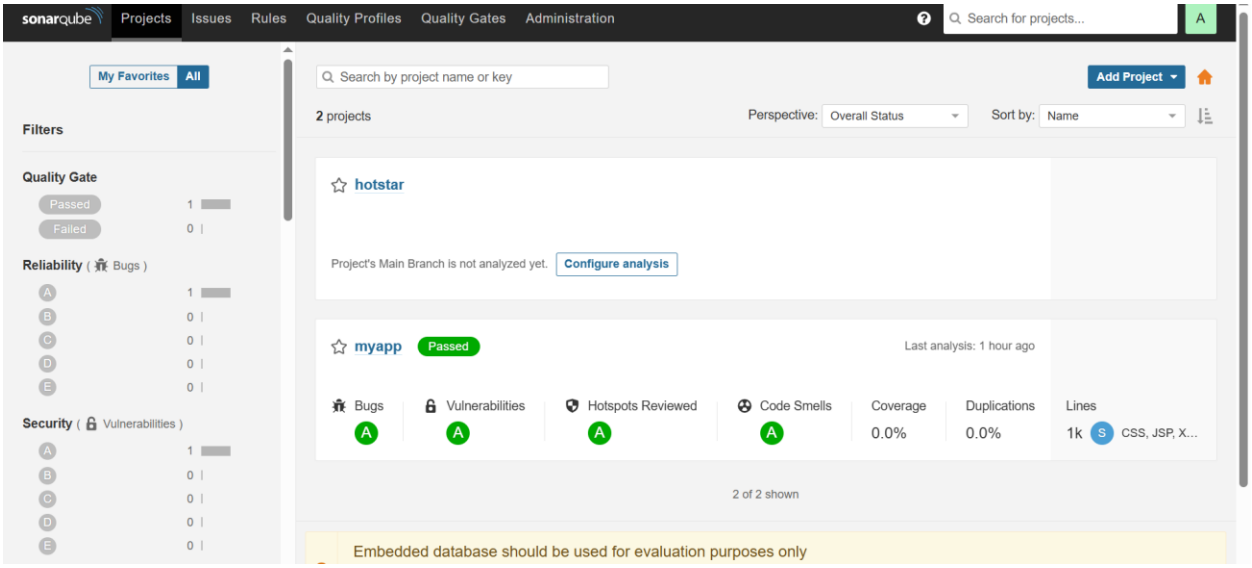


Fig 5: SonarQube project dashboard — "myapp" passing the Quality Gate with an "A" rating across all metrics

6.4 Deployed Application

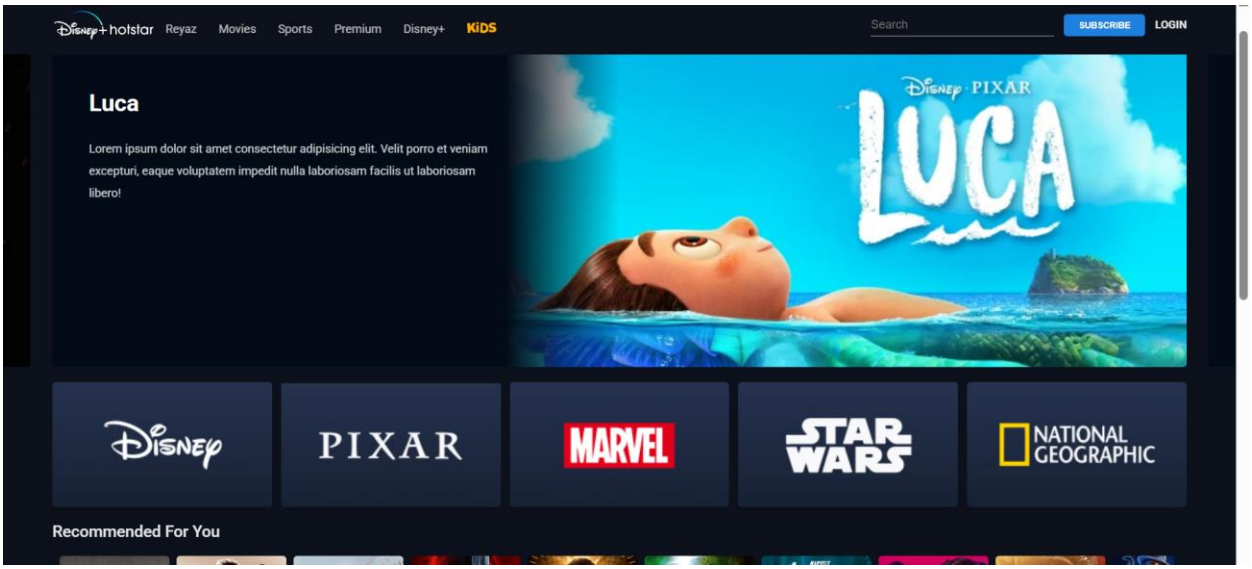


Fig 6: Application deployed and served from Apache Tomcat on AWS EC2

7. Deployment Details

Component	Endpoint
Deployed Application (Tomcat / EC2)	http://15.206.93.1:8080/mywebapp/
Jenkins Server	http://13.126.203.227:8080/
SonarQube Server	http://13.207.150.164:9000/

8. Conclusion

This project demonstrates a complete, automated CI/CD workflow for a Java web application: source code is built with Maven, validated against code-quality standards with SonarQube, and deployed to a live Tomcat server on AWS — all orchestrated end-to-end by a single Jenkins pipeline. The setup illustrates key DevOps practices including pipeline-as-code, automated quality gating, and cloud-based deployment, and provides a reusable template that can be extended with additional stages such as containerisation or automated rollback.